# Big-O Notation

شرح Big-O Notation ببساطة ❤

🌟 الـ Big-O Notation هي طريقة بنستخدمها في علم الكمبيوتر علشان نقيس أداء الأكواد، وخصوصًا مدى سرعتها لما البيانات تكبر. الفكرة إننا بنحسب أسوأ سيناريو للأداء علشان نعرف الكود هيتصرف إزاي لما يكون عندنا بيانات ضخمة.

♦ ليه نستخدم Big-O؟

لما تيجي تختار خوارزمية، لازم تفكر:
✅ هل هتشتغل بسرعة مع بيانات كبيرة؟
✅ هل استهلاكها للموارد معقول؟

🌟 الـ Big-O بيساعدك تفهم دا من غير ما تجرب الكود عمليًا على كل الأحجام الممكنة من البيانات.

♦ أشهر أنواع Big-O:

1️⃣ O(1) - Constant Time (وقت ثابت)

الخوارزمية بتشتغل في نفس الوقت مهما كبرت البيانات.
💡 مثال: الوصول إلى عنصر معين في Array arr[5]

2️⃣ O(log n) - Logarithmic Time (لوغاريتمي)

الأداء بيزيد ببطء جدًا مع زيادة البيانات.
💡 مثال: البحث الثنائي (Binary Search)

3️⃣ O(n) - Linear Time (خطّي)

الأداء بيتناسب طرديًا مع عدد العناصر.
💡 مثال: البحث في Array عنصر بعنصر

4️⃣ O(n log n) - Quasilinear Time

أداء أفضل من $O(n^2)$ لكنه أبطأ من $O(n)$.
💡 مثال: خوارزميات الفرز الفعّالة زي Merge Sort و Quick Sort

5️⃣ $O(n^2)$ - Quadratic Time (تربيعي)

الأداء بيزيد بشكل كبير لما البيانات تكبر.
💡 مثال: خوارزميات الترتيب السيئة زي Bubble Sort

6️⃣ $O(n^3)$ - Cubic Time (تكعيبي)

الأداء بيزيد بشكل ضخم جدًا لما البيانات تكبر.
💡 مثال: مصفوفات الـ Brute Force لما بتضرب 3 حلقات متداخلة (Nested Loops)

### 7️⃣ O(2ⁿ) - Exponential Time (أسيّ)

الوقت بيزيد بشكل جنوني مع زيادة البيانات!

💡 مثال: توليد جميع التوافيق الممكنة لمجموعة عناصر

### 8️⃣ O(n!) - Factorial Time (عاملي)

ده الأسوأ، يعني مستحيل تشتغل بيه على بيانات كبيرة.

💡 مثال: حل مسألة Traveling Salesman بطريقة Brute Force

♦ ازاي تختار خوارزمية كويسة؟

✅ حاول دايمًا تشتغل على O(1) أو O(log n) لو أمكن
✅ تجنب O(n²) أو الأسوأ، خصوصًا لو البيانات كبيرة
✅ افهم المشكلة قبل ما تختار الخوارزمية

🎯 الخلاصة:

🌟 الـ Big-O Notation هي وسيلة لتقييم أداء الأكواد بدون تشغيلها على كل السيناريوهات. لما تفهمها، تقدر تكتب أكواد أسرع وأكثر كفاءة.

| Big-O | Name | Description | Example |
|---|---|---|---|
| **O(1)** | **Constant Time** | Execution time remains unchanged irrespective of input data | Checking if a stack is empty |
| **O(log n)** | **Logarithmic Time** | Complexity increases by one unit for each doubling of input data | Finding an item in a balanced search tree |
| **O(n)** | **Linear Time** | Execution time increases linearly with the size of the input data | Linear traversal of a list |
| **O(n log n)** | **Log-Linear Time** | Complexity grows as a combination of linear and logarithmic | Merge sort on a collection of items |
| **O(n^2)** | **Quadratic Time** | Time taken is proportional to the square of the number of elements | Checking all possible pairs in an array |
| **O(n^3)** | **Cubic Time** | Execution time is proportional to the cube of the number of elements | Matrix multiplication of n x n matrices |
| **O(2^n)** | **Exponential Time** | Time doubles for every new element added | Generating all subsets of a given set |
| **O(n!)** | **Factorial Time** | Complexity grows factorially based on the size of the dataset | Determining all permutations of a given list |